

Multilabel Classification Networks

Model, Learning, Evaluation, and Threshold Tuning

Yasser Hifny

Faculty of Computers and Artificial Intelligence
Capital University

Multilabel Classification

- In multilabel classification, an instance may belong to multiple classes simultaneously.
- Example: a document may have the topics *politics*, *religion*, and *education*.
- This differs from multiclass classification, where each instance belongs to one class.
- The output is represented by a **multi-hot vector**.

Key idea

Each output label is treated as a binary decision, allowing several labels to be active at the same time.

Example: Movie Genres

A movie can belong to several genres. For three possible genres:

$$\mathbf{t} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad \text{Comedy only}$$

$$\mathbf{t} = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} \quad \text{Romance + Action}$$

$$\mathbf{t} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \quad \text{No genre}$$

Multi-hot representation

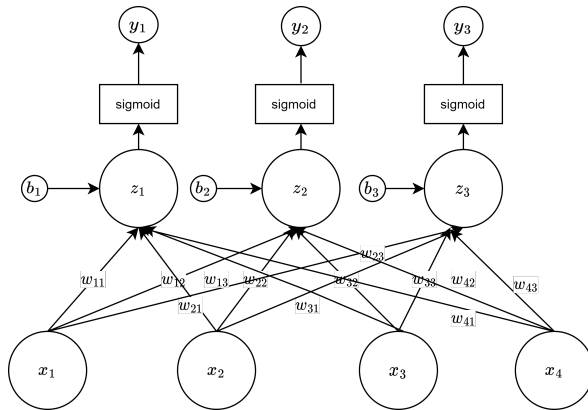
Each component is either 0 or 1 and indicates whether the corresponding label is present.

For $L = 3$ labels:

$$2^3 = 8$$

possible multi-hot outputs exist.

Multilabel Network



Input with 4 variables and 3 independent output labels.

Multilabel Model

For L labels, the model contains one sigmoid output for each label:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_L \end{bmatrix} = \begin{bmatrix} \sigma(\mathbf{w}_1^T \mathbf{x} + b_1) \\ \sigma(\mathbf{w}_2^T \mathbf{x} + b_2) \\ \vdots \\ \sigma(\mathbf{w}_L^T \mathbf{x} + b_L) \end{bmatrix}.$$

- $\mathbf{x}$: input vector.
- $\mathbf{w}_l$: weight vector for label l .
- b_l : bias for label l .
- $\sigma(\cdot)$: sigmoid function.

Why Sigmoid Rather Than Softmax?

Multilabel

- Labels are nonexclusive.
- Each label has its own probability.
- Several probabilities can be high simultaneously.
- Uses independent sigmoid outputs.

Multiclass

- Classes are mutually exclusive.
- Output probabilities are coupled.
- One class is selected.
- Uses softmax output.

Multilabel $\implies L$ independent binary decisions

Learning Problem

The objective is the sum of the binary cross-entropy losses for all labels.

For one label:

$$E(t, y) = -t \log(y) - (1 - t) \log(1 - y).$$

For L labels:

$$E(\mathbf{t}, \mathbf{y}) = - \sum_{l=1}^L [t_l \log(y_l) + (1 - t_l) \log(1 - y_l)]$$

- $t_l \in \{0, 1\}$ is the target for label l .
- $y_l \in (0, 1)$ is the predicted probability.

Learning Algorithms: Comparison

Model	Linear Regression	Binary	Multiclass	Multilabel
Activation	Linear	Sigmoid	Softmax	Sigmoid
Output nodes	$K \geq 1$	$K = 1$	$K \geq 3$	$K \geq 2$
Objective	MSE	Binary CE	Categorical CE	K Binary CE

Distinction

The multilabel model uses multiple sigmoid outputs and a binary cross-entropy term for every label.

Evaluation: Micro F1

Micro F1 aggregates the contributions of all labels before computing precision, recall, and F1.

$$P_{\text{micro}} = \frac{\sum_{l=1}^L TP_l}{\sum_{l=1}^L (TP_l + FP_l)}$$

$$R_{\text{micro}} = \frac{\sum_{l=1}^L TP_l}{\sum_{l=1}^L (TP_l + FN_l)}$$

$$F1_{\text{micro}} = \frac{2P_{\text{micro}}R_{\text{micro}}}{P_{\text{micro}} + R_{\text{micro}}}$$

It gives each true/false positive and negative an equal contribution to the aggregate score.

Micro F1: Components

- **True Positive (TP)**: label correctly predicted as present.
- **False Positive (FP)**: label incorrectly predicted as present.
- **False Negative (FN)**: label incorrectly predicted as absent.

For labels $\{1, \dots, L\}$ across N samples, the counts are aggregated across labels.

Interpretation

Micro averaging emphasizes the aggregate prediction behavior over all labels and instances.

Micro F1: Numerical Example

For three labels:

Label	<i>TP</i>	<i>FP</i>	<i>FN</i>
1	10	5	3
2	20	4	6
3	15	3	7

$$P_{\text{micro}} = \frac{45}{45 + 12} = \frac{45}{57} \approx 0.789$$

$$R_{\text{micro}} = \frac{45}{45 + 16} = \frac{45}{61} \approx 0.738$$

$$F1_{\text{micro}} = \frac{2(0.789)(0.738)}{0.789 + 0.738} \approx 0.762$$

Decision Threshold

Each sigmoid output produces a score s_l for label l .

$$\hat{y}_l = \begin{cases} 1, & s_l \geq \tau_l, \\ 0, & s_l < \tau_l. \end{cases}$$

- The common default is $\tau_l = 0.5$.
- This default threshold need not maximize Micro F1.
- Thresholds can be tuned using validation data.
- A separate threshold can be selected for each label, or a single global threshold can be used.

Threshold Tuning

Threshold tuning changes the balance between false positives and false negatives.

Per-label tuning

Search for

$$\{\tau_1, \tau_2, \dots, \tau_L\}$$

and evaluate the desired metric for each candidate set.

- Grid search is one possible method.
- The validation set can be used to select thresholds.
- The objective may be Micro F1, precision, recall, or another application-specific metric.

Probability output $\xrightarrow{\text{threshold}}$ binary label

Macro F1

Macro F1 computes F1 independently for each label and then averages the results.

For label l :

$$F1_l = \frac{2P_lR_l}{P_l + R_l}.$$

The macro score is:

$$F1_{\text{macro}} = \frac{1}{L} \sum_{l=1}^L F1_l$$

with

$$P_l = \frac{TP_l}{TP_l + FP_l}, \quad R_l = \frac{TP_l}{TP_l + FN_l}.$$

Micro F1 vs. Macro F1

Micro F1

- Aggregates TP, FP, and FN across labels.
- Frequent labels contribute more observations.
- Describes global prediction performance.

Macro F1

- Computes one F1 per label.
- Gives every label equal weight.
- Provides a balanced view across labels.

Evaluation strategy

We use Micro F1, Macro F1, and threshold tuning together for multilabel evaluation.

Python Example

Model structure

The example implements a linear multilabel classifier with:

- n input variables,
- L sigmoid outputs,
- trainable weights and biases,
- iterative parameter updates.

The complete implementation from the source is shown on the following slides.

Python Example: Model

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

class LinearNN:
    def __init__(self, n_inputs, n_outputs):
        self.weights = np.random.rand(n_inputs, n_outputs)
        self.bias = np.random.rand(n_outputs)

    def train(self, X, T, epochs, lr):
        for epoch in range(epochs):
            y_pred = self.predict(X)

            d_weights = np.dot(
                X.T,
                (y_pred - T) * sigmoid_derivative(y_pred)
            )
            d_bias = np.sum(
                (y_pred - T) * sigmoid_derivative(y_pred),
                axis=0
            )

            self.weights -= lr * d_weights
```

Python Example: Prediction and Training

```
def predict(self, X):  
    return sigmoid(np.dot(X, self.weights) + self.bias)  
  
if __name__ == "__main__":  
    X = np.array([[0, 0, 1],  
                  [0, 1, 1],  
                  [1, 0, 1],  
                  [1, 1, 1]])  
  
    T = np.array([[0, 0, 1],  
                  [1, 0, 0],  
                  [0, 1, 0],  
                  [1, 1, 1]])  
  
    model = LinearNN(X.shape[1], T.shape[1])  
    model.train(X, T, epochs=1000000, lr=0.001)  
  
    print("Model weights:")  
    print(model.weights)  
    print("Model biases:")  
    print(model.bias)
```

Python Example: Learning Flow

$$\mathbf{X} \xrightarrow{\mathbf{W}, \mathbf{b}} \mathbf{z} \xrightarrow{\sigma} \mathbf{Y}$$

$$\mathbf{Y}, \mathbf{T} \xrightarrow{\text{loss}} E \xrightarrow{\text{gradient}} \nabla_{\mathbf{W}} E, \nabla_{\mathbf{b}} E$$

$$\boxed{\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} E, \quad \mathbf{b} \leftarrow \mathbf{b} - \eta \nabla_{\mathbf{b}} E}$$

The source example repeats this process for the specified number of epochs.

Dataset Used in the Example

The source example uses:

$$X = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

and target matrix

$$T = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}.$$

- Four training instances.
- Three input variables.
- Three output labels.
- The last column of X is constant and acts as an input bias feature in addition to the model bias.

Assignment

Implement a multilabel classification model using Python from scratch, using the Keras deep learning library, and Google Colab.

Dataset:

`multilabel-classification-dataset`

We will use the following Kaggle dataset URL:

<https://www.kaggle.com/datasets/shivanandmn/multilabel-classification-dataset>

Suggested evaluation

Report multilabel performance using Micro F1 and Macro F1, and investigate the effect of decision-threshold tuning.

Summary

- Multilabel classification assigns zero, one, or several labels to an instance.
- Targets are represented using multi-hot vectors.
- Each label uses an independent sigmoid output.
- The learning objective is the sum of binary cross-entropies.
- Micro F1 evaluates aggregate label-level performance.
- Macro F1 gives equal weight to every label.
- Decision thresholds can be tuned using validation data.
- We provide a NumPy implementation and a Keras-based assignment.

Thank You

Questions?